

The whole-system technical paper

Contents

Contents	1
Status	2
The problem	2
Why it matters	2
What was built	2
What was deliberately NOT built	3
How it was proved	3
Open questions	3
Addendum, 25 September: PDF output	3

Status

Draft complete end to end. Not submitted. Venue not chosen. 25 September 2026.

Commits: 6e3d757, and the Part II / Part III / Part IV commits that follow it. Everything is in `paper/`.

Supersedes `2026-09-24-technical-paper-draft.md`, which recorded a paper on a single finding.

The problem

The paper written on 24 September took one thread — the false-absence defect class — and made it the whole subject. The owner read it and said, correctly, that it did not show what had been accomplished:

"I wanted to do a paper for the whole concept of the whole project, not picking one part or maybe a failure. Everything should be the whole concept, the whole of what has been achieved, what is done, layer by layer."

The earlier choice was defensible as publishing strategy and wrong as a record of the work. A single-defect study is the most reviewable claim available from this project; it is not the project. It also had the effect of presenting three months of building as a backdrop to one failure.

Why it matters

The design records exist so that this project can be written up without reconstructing it from memory. If the write-up they feed covers one eighth of the work, the records were kept for nothing. And the parts left out were the parts with the most transferable content — the knowability tiers, the boot chain, the erasure remediation, the deletions.

What was built

`paper/` restructured, and a nineteen-section paper written into it.

Title	<i>Proving what a second-hand computer is: an evidence-first architecture for IT asset disposition</i>
Length	16,178 words of prose, 29 tables and figures
Structure	Part I the problem (§§1–3), Part II the layers (§§4–10), Part III what it taught (§§11–14), Part IV closing (§§15–19)
Assembly	<code>paper/sections/NN-*.md</code> → <code>PAPER.md</code> via <code>assemble.py</code>

The spine is a thesis the build actually followed from week one, rather than a feature list:

Every output of this system is a claim about a machine the system does not control. Its entire value is how honestly it separates what it has established from what it has merely assumed.

That is what makes the offline queue, the boot stick, the registry reader, the wipe ladder, the certificate and the Autopilot limit one system rather than seven projects.

Three findings are positioned as the contribution: the four knowability tiers (§11), the false-absence class and its 32 call sites (§12), and the recurrence result — naming the class prevented nothing (§12.4).

The earlier draft was not discarded. It is intact in `paper/archive/` and is the full treatment of what is now §12, compressed from ~3,700 words to ~1,300. The README says it should be offered as a companion artefact rather than folded back in.

What was deliberately NOT built

Not a feature list. A layer-by-layer catalogue with no organising argument would be unpublishable and useless to a reader in six months. Every layer section states what was rejected as well as what was built.

No production telemetry was invented. §15.2 says outright that no throughput or field-error figures exist, because none were collected. The temptation to estimate them was the single largest in the draft.

§18 was not written as if the citations were checked. They are positioned from knowledge of those literatures, not the sources in hand. The section and the README both say it is not submittable in that state.

The word target was exceeded and not enforced. The outline said 12,000–15,000; it came out at 16,178. Cutting to a self-imposed target would have meant removing layer coverage, which was the entire point of the rewrite.

How it was proved

Figures measured from the repository, not recalled: 535 commits, ~101,000 lines across four trees, 23 entities, 62 migrations, 30 pages, 23 controllers, 96 test files, 40 design records, one engineer (three git identities, all the same address or a typo of it).

Two figures in §13 were wrong in draft 1 and were corrected from the commit history rather than the design records, which were loose on both: warranty tracking was present in the initial commit of 9 July, not built on the 11th, and the repairs module lasted 39 days rather than "six weeks". Noted in that section's drafting notes.

Three figures in §10 were verified before the section was committed — ten functional defects found by the visual-system pass, thirty-one pages, eleven reporting slices — because the previous paper asserted three figures before measuring them and was wrong each time.

Cross-checks run: §17's thesis paragraph is word-for-word identical to §1.1's. `assemble.py` regenerates `PAPER.md` and reports prose and tables separately, so no word count is ever asserted in prose.

Open questions

1. **§18's citations are the one hard blocker.** Nothing should be submitted until each is checked against its source. The section's notes list what to check.
2. **Is there a prior naming of the false-absence class?** §18.5 states the framing conditionally because the search is incomplete.
3. **Venue.** arXiv/Zenodo as written is the recommendation; ICSE SEIP or ESEM would need Part II cut. A short version should be written *from* this, never by reducing it.
4. **A second rater on ten random instances** answers §16.2, the most serious threat in the paper. None available so far.
5. Whether the business is named or anonymised.
6. Several drafting notes name a specific pre-submission cross-check that has not been run — notably verifying §15.3's evidence table against each design record's "How it was proved" heading.

Addendum, 25 September: PDF output

The owner could not read the `.md` files comfortably and asked for PDFs.

What was built: `tools/md-to-pdf.py`, typesetting the repo's Markdown with reportlab — already a dependency for the operator guides, so no LaTeX, no pandoc, no headless browser. Handles headings, pipe tables, fenced code, blockquotes, lists and task lists, with a clickable contents page for anything with six or more sections. Output in `paper/pdf/`: the paper at **37 pages**, plus the README and outline.

What was also built, and mattered: `tools/verify-pdf.py`, which checks the PDF against its source — every heading present and in order, every table cell and code line present, and no text past the margin.

It was written because a converter that silently drops a table is the defect class this project keeps finding: the build succeeds, the output looks finished, and something is missing. **It found a real defect on first use.** Inline code was rendered at a fixed 8.6pt; inside a 15pt heading that put the fragment on its own baseline and split the line in two. Nothing in the build complained. Code now inherits the surrounding size.

Two of my own bugs on the way, both recorded because they are the same shape as the ones in the paper:

1. Regex substitution for emphasis produced *overlapping* tags on ***bold with italic ending***, which reportlab rejects outright. Emphasis is now tokenised with a stack that cannot emit unbalanced markup, and the tokeniser was unit-checked against six cases before use.
2. The first two versions of the verifier reported false alarms — wrapped table cells read as missing, and the running footer's repeated title read as a heading out of order. Both were fixed in the *checker*, not waved away, and only then did the one genuine defect stand out.

The second is worth keeping in view: a verification tool with a high false-alarm rate is a tool whose real findings get ignored.

Verified: all three documents report VERDICT: `clean`. The architecture diagram renders at 8pt with its widest point at 447pt against a 544pt margin.